Terraform Modules

To Begin with the Labs

Summary of the Lab

This lab focuses on creating and using reusable Terraform modules to simplify AWS infrastructure deployment. First, a Terraform module stored in GitHub is used to provision an EC2 instance by supplying only essential values such as the AMI ID, instance type, subnet ID, and tags, while securely managing AWS credentials through environment variables. Next, a custom local VPC module is developed to create reusable networking components with configurable settings, allowing separate Development and Development-Q environments to be deployed with different CIDR ranges, IPv6 configurations, regions, and tags using the same code base.

The lab then expands the custom module into a complete network module that provisions a VPC, Internet Gateway, Public Subnet, Route Table, Route Table Association, and Security Group. Input variables are used to make the module flexible and reusable, while outputs expose important resource IDs for use by other Terraform configurations.

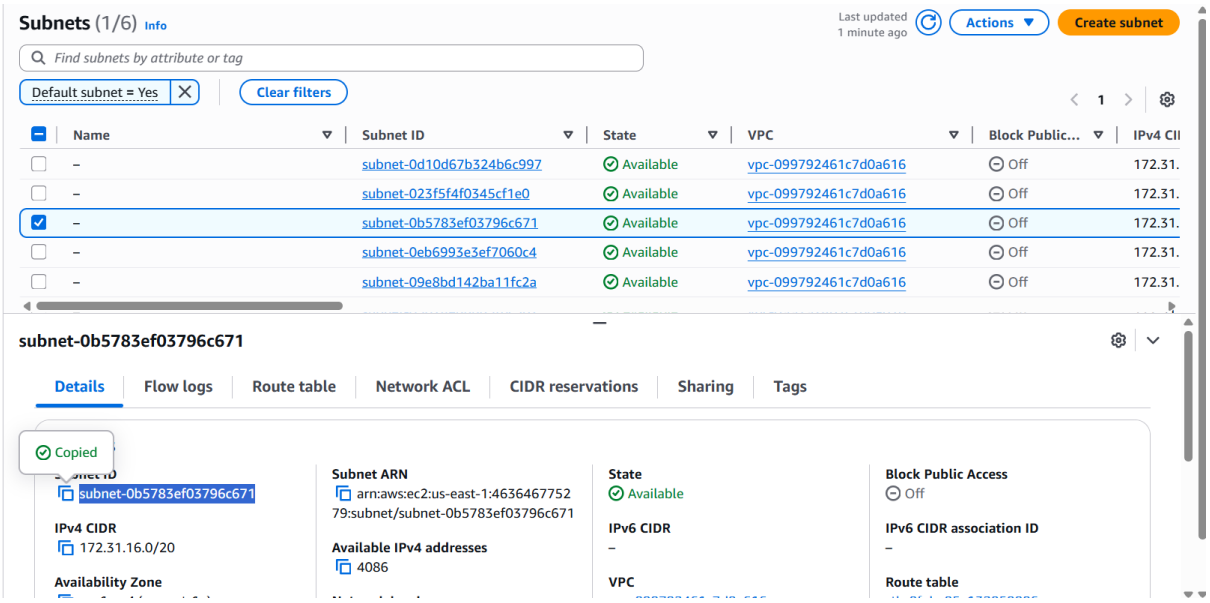
Finally, the network module is consumed by a root Terraform configuration that creates an AWS Key Pair and launches an EC2 instance inside the custom VPC. The instance automatically uses the subnet and security group created by the module through Terraform outputs. After deployment, all resources are verified in the AWS Console, confirming successful creation of the VPC, networking components, security configurations, and EC2 instance. Overall, the lab demonstrates how Terraform modules promote code reuse, consistency, easier maintenance, and scalable infrastructure deployments across multiple environments.

- Create 2 files:
- main.tf
- provider.tf
- Create a Terraform module block.
- Specify the GitHub repository URL as the module source.
- Remove https:// from the GitHub URL if required by the module source format.
- Define the mandatory values required by the module:
- EC2 instance name
- AMI ID
- Instance type
- Subnet ID
- Tags
- Example values used:
- AMI ID from the desired AWS region
- Instance type: t2.micro
- Subnet ID from the target VPC subnet

- Tags such as:
- Environment = Dev
- Terraform = True

```
  GNU nano 8.7.1                                                main.tf *
module "ec2_cluster" {
    source = "github.com/terraform-aws-modules/terraform-aws-ec2-instance.git"

    name    = "my-cluster"
    ami     = "ami-0f40c8f97004632f9"
    instance_type          = "t2.micro"
    subnet_id   = "subnet-0b5783ef03796c671"

    tags = {
    Terraform   = "true"
    Environment = "dev"
    }
}
```

- Open AWS Console.
- Navigate to VPC → Subnets.
- Select a subnet in the same AWS region as the AMI.
- Copy the Subnet ID.



- Export AWS credentials as environment variables.

```
export AWS_ACCESS_KEY_ID="YOUR_ACCESS_KEY_ID"

export AWS_SECRET_ACCESS_KEY=<your-secret-key>

echo $AWS_ACCESS_KEY_ID

echo $AWS_SECRET_ACCESS_KEY
```

when you run the command

```
terraform plan
```

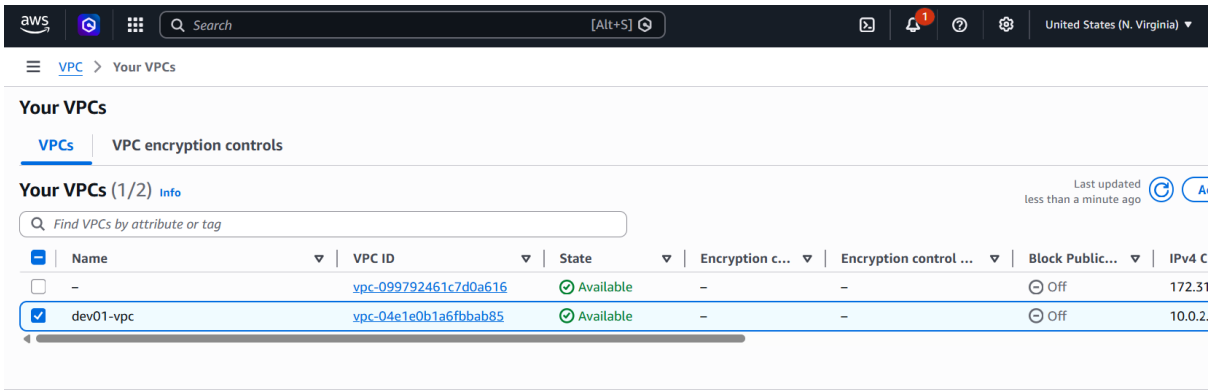
this will create 4 resources.

```
    }

Plan: 4 to add, 0 to change, 0 to destroy.
```

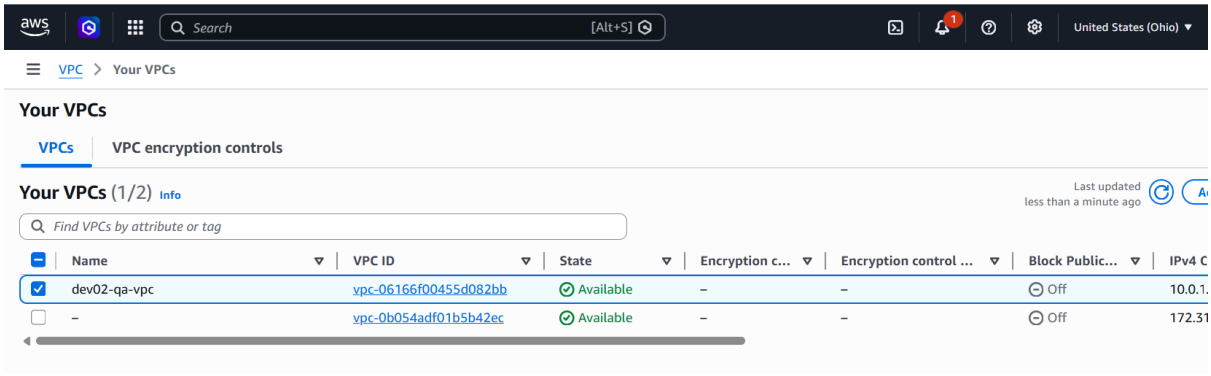
- Create a reusable Terraform module to provision an AWS VPC with configurable settings.
- Define the VPC resource in vpc.tf using variables for VPC name, CIDR block, instance tenancy, DNS settings, ClassicLink options, IPv6 assignment, and environment tags.
- Declare all required input variables in variables.tf, including AWS credentials, region, networking parameters, and environment details, with default values where appropriate.
- Configure the AWS provider in provider.tf and make the AWS region configurable through a variable.
- Create development/main.tf to call the local module with Development-specific values such as dev-01-vpc, CIDR 10.0.2.0/24, IPv6 enabled, and environment Development Engineering.
- Create development_q/main.tf to reuse the same module with values such as dev-q-02-vpc, CIDR 10.0.1.0/24, IPv6 disabled, region us-east-1, and environment Development-Q Engineering.

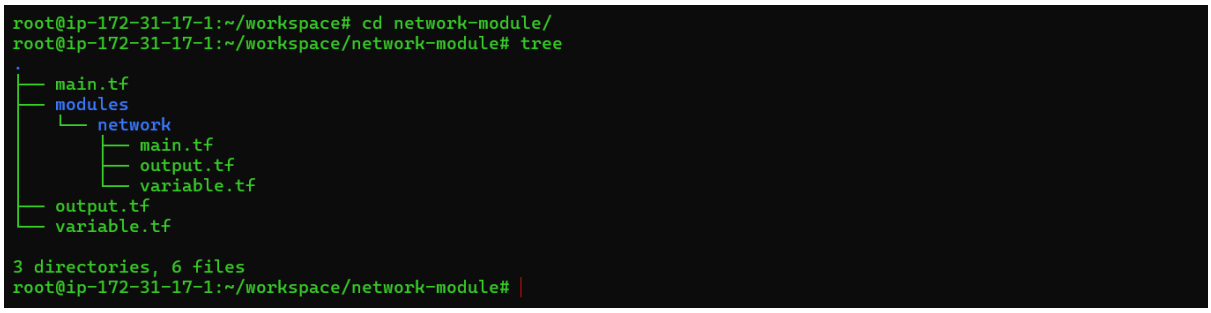
```
root@ip-172-31-17-1:~/workspace/terraform-module# tree
.
├── Development
│   └── main.tf
├── Development_QA
│   └── main.tf
├── provider.tf
├── variable.tf
└── vpc.tf

3 directories, 5 files
root@ip-172-31-17-1:~/workspace/terraform-module#
```

- Verify the Development Environment VPC in AWS Console
  - Region: us-east-1
  - CIDR Block: 10.0.2.0/24
  - IPv6: Enabled
  - Tags: dev-01-vpc, Development Engineering



- Verify the Development-Q Environment VPC in AWS Console
  - Region: us-east-2
  - CIDR Block: 10.0.1.0/24
  - IPv6: Disabled
  - Tags: dev-q-02-vpc, Development-Q Engineering


aws
Search
[Alt+S]
United States (Ohio)
VPC
Your VPCs
Your VPCs
VPCs
VPC encryption controls
Your VPCs (1/2) Info
Last updated
less than a minute ago
Find VPCs by attribute or tag
Name
VPC ID
State
Encryption c...
Encryption control ...
Block Public...
IPv4 C
dev02-qa-vpc
vpc-06166f00455d082bb
Available
–
–
Off
10.0.1.
–
vpc-0b054adf01b5b42ec
Available
–
–
Off
172.31



- Create the VPC and Internet Gateway
- Define an aws_vpc resource with:
    - VPC CIDR block
    - enable_dns_support = true
    - enable_dns_hostnames = true
    - Environment tag
- Create an aws_internet_gateway resource and attach it to the VPC.
- Add the same Environment tag.
- Create a Public Subnet
- Define an aws_subnet resource with:
    - Subnet CIDR block
    - Availability Zone
    - map_public_ip_on_launch = true (auto-assign public IPs)
    - VPC ID reference
    - Environment tag
- Create Routing Components
- Create an aws_route_table for the VPC.
- Add a route:
    - Destination: 0.0.0.0/0
    - Target: Internet Gateway
- Add an Environment tag.
- Create an aws_route_table_association to associate the public subnet with the public route table.
- Create a Security Group
- Define an aws_security_group in the VPC with:
    - Inbound rule: TCP port 22 (SSH) from 0.0.0.0/0
    - Outbound rule: All protocols, all ports to 0.0.0.0/0
- Add an Environment tag to the security group.
- Define input variables in variables.tf for all configurable values used by the module:
    - vpc_cidr_block → default: 10.1.0.0/16
    - subnet_cidr_block → default: 10.1.0.0/20
    - availability_zone → default: us-east-2a
    - public_key_name → default: levelup_key.pub

    - environment → default: production
- Use these variables throughout the Terraform configuration instead of hard-coded values to make the infrastructure reusable, configurable, and easier to manage across different environments.
- Define outputs in outputs.tf for the key networking resources created by the module:
    - vpc_id → Outputs the VPC ID
    - public_subnet_id → Outputs the Public Subnet ID
    - security_group_id → Outputs the Security Group ID
- Use these outputs for downstream resources, such as EC2 instances, so other Terraform configurations or modules can reference the created VPC, subnet, and security group without hard-coding resource IDs.
- Create main.tf
    - Configure the AWS provider.
    - Reference the custom network module (./module/network).
    - Create an AWS Key Pair.
    - Launch an EC2 instance.
- Configure the AWS Provider
    - Define the aws provider.
    - Read the AWS region from a variable (e.g., us-east-2).
- Use the Custom Network Module
    - Create a module block named network.
    - Set the source to ./module/network to use the local networking module.
- Create an AWS Key Pair
    - Define an aws_key_pair resource.
    - Use a key name (e.g., levelup_key).
    - Read the public key file path from a Terraform variable.
- Create an EC2 Instance
    - Define an aws_instance resource.
    - Configure the AMI ID, instance type, key pair, subnet, security group, and Environment tag.
- Use Module Outputs
    - Set subnet_id using module.network.public_subnet_id.
    - Set vpc_security_group_ids using module.network.sg_22_id.
    - This reuses networking resources created by the module.
- Create variables.tf
    - Define variables for:
        - region (e.g., us-east-2)
        - public_key_path (e.g., levelup_key.pub)
        - ami (Amazon Linux AMI)
        - instance_type (e.g., t2.micro)
        - environment (e.g., production)
- Create outputs.tf
    - Add an output named instance_public_ip.

    - Display the EC2 instance's public IP address for easy SSH access after deployment.
- Navigate to the SSH directory and generate the key pair

```
cd /root/.ssh

ssh-keygen -f levelup_key
```

- Press Enter to accept the default prompts and create the key pair.
- Verify the generated files
    - Private key: /root/.ssh/levelup_key
    - Public key: /root/.ssh/levelup_key.pub
- The public key file (levelup_key.pub) should match the path specified in your Terraform variable (public_key_path).

```
root@ip-172-31-17-1:~/workspace/network-module# cd /root/.ssh
root@ip-172-31-17-1:~/.ssh# ls
authorized_keys  known_hosts  known_hosts.old
root@ip-172-31-17-1:~/.ssh# ssh-keygen -f levelup_key
Generating public/private ed25519 key pair.
Enter passphrase for "levelup_key" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in levelup_key
Your public key has been saved in levelup_key.pub
The key fingerprint is:
SHA256:Rza/NvbyQFwdBknkwdPtwlUt9m4hZ5Rxnx87r69SogE root@ip-172-31-17-1
The key's randomart image is:
+--[ED25519 256]--+
|            +=++O|
|            .+==O|
|          +  ++Bo|
|        Eo + oo+*|
|        S.. + +=o|
|         ..... =|
|           o*o ..|
|          .oo+ . |
|             +=o.|
+----[SHA256]-----+
root@ip-172-31-17-1:~/.ssh#
```

- Move to the Terraform network module directory.

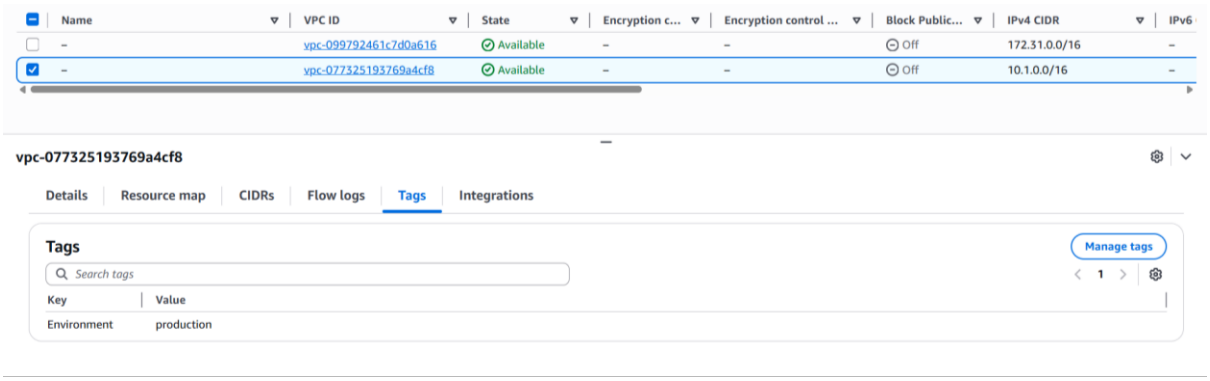
```
cd ~/<folder name>/network-module
```

- Initialize Terraform
- Generate the Terraform Execution Plan

```
Terraform apply
```

```
module.network.aws_vpc.levelup_vpc: Creating...
aws_key_pair.levelup_key: Creating...
aws_key_pair.levelup_key: Creation complete after 1s [id=levelup_key]
module.network.aws_vpc.levelup_vpc: Creation complete after 2s [id=vpc-077325193769a4cf8]
module.network.aws_internet_gateway.levelup_igw: Creating...
module.network.aws_subnet.subnet_public: Creating...
module.network.aws_security_group.levelup_sg_22: Creating...
module.network.aws_internet_gateway.levelup_igw: Creation complete after 1s [id=igw-0694f5ea9e17a1eff]
module.network.aws_route_table.levelup_rtb_public: Creating...
module.network.aws_route_table.levelup_rtb_public: Creation complete after 1s [id=rtb-01320722f1614b67b]
module.network.aws_security_group.levelup_sg_22: Creation complete after 3s [id=sg-031b9738042ae0ee6]
module.network.aws_subnet.subnet_public: Still creating... [00m10s elapsed]
module.network.aws_subnet.subnet_public: Creation complete after 11s [id=subnet-064f2b45f8767dc90]
module.network.aws_route_table_association.subnet_public_assoc: Creating...
aws_instance.levelup_instance: Creating...
module.network.aws_route_table_association.subnet_public_assoc: Creation complete after 1s [id=rtbassoc-0a947ed14d9cd16ca]
aws_instance.levelup_instance: Still creating... [00m10s elapsed]
aws_instance.levelup_instance: Still creating... [00m20s elapsed]
aws_instance.levelup_instance: Still creating... [00m30s elapsed]
aws_instance.levelup_instance: Creation complete after 33s [id=i-0fd10baf53b81e51b]

Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

public_instance_ip = "54.92.219.24"
```

- In the AWS Console, go to VPC → Your VPCs and verify that the custom VPC exists with the Environment tag set to Production and the CIDR block configured as 10.1.0.0/16.



- In the AWS Console, navigate to EC2 → Instances and verify that the instance is Running, uses the t2.micro instance type, the correct AMI, the levelup_key key pair, has a private IP address from the custom subnet, and is associated with the levelup_sg_22 security group.